9. Travelling Salesman Problem (TSP)

```
from itertools import permutations

def travelling_salesman_problem(graph, start):
    vertices = list(graph.keys())
    vertices.remove(start)

    min_path_weight = float('inf')
    best_path = []

    # Generate all permutations of remaining cities
    for perm in permutations(vertices):
        current_path_weight = 0
        current_node = start
        valid_path = True

        # Calculate weight of the current path permutation
        for next_node in perm:
            if next_node in graph[current_node]:
                current_path_weight += graph[current_node][next_node]
                current_node = next_node
            else:
                valid_path = False
                break

        # Complete cycle by returning back to start
        if valid_path and start in graph[current_node]:
            current_path_weight += graph[current_node][start]
            if current_path_weight < min_path_weight:
                min_path_weight = current_path_weight
                best_path = [start] + list(perm) + [start]
```

```
    return min_path_weight, best_path

if __name__ == "__main__":
    # Distance matrix represented as an adjacency list dictionary
    distance_graph = {
        'A': {'B': 10, 'C': 15, 'D': 20},
        'B': {'A': 10, 'C': 35, 'D': 25},
        'C': {'A': 15, 'B': 35, 'D': 30},
        'D': {'A': 20, 'B': 25, 'C': 30}
    }

    cost, route = travelling_salesman_problem(distance_graph, 'A')
    print(f"Minimum TSP cost: {cost}")
    print(f"Optimal Route: {' -> '.join(route)}")
```

